Mini-Rapport – Projet 6

Système de Coordination des Soins à Domicile

Auteurs :	Costo Aya & Lagribi Abdelilah
Filière :	INDIA 2ème année – S4 | ENSAM Rabat
A.U. :	2025/2026 | Pr. A. El Qadi
Date :	Avril 2026

1. Présentation du Projet

Ce projet vise à développer un **Système de Coordination des Soins à Domicile** basé sur une architecture microservices. L'objectif est d'optimiser la planification des visites des soignants à domicile en tenant compte des contraintes géographiques, temporelles et cliniques, tout en assurant la traçabilité et la facturation des prestations.

Acteurs du système

- **Patients à domicile / Aidants** : bénéficiaires des soins
- **Infirmiers libéraux / Aides-soignants** : prestataires de soins à domicile
- **Coordinateurs de soins / Planificateurs** : gestionnaires des tournées
- **Médecins prescripteurs** : ordonnancement et prescription des soins

2. Architecture Microservices

L'application est décomposée en **5 microservices métier**, un **API Gateway** et un **serveur de découverte Eureka**. Chaque service est indépendant, possède sa propre base de données (pattern Database per Service) et est conteneurisé.

Service	Port	Rôle
Eureka Server	8761	Service Discovery – enregistrement et découverte dynamique
API Gateway	8080	Point d'entrée unique, routage, load balancing
patient-home-service	8081	Profil patient à domicile, besoins, contraintes, géolocalisation
caregiver-scheduling-service	8082	Planning des soignants, disponibilités, compétences
visit-optimization-service	8083	Algorithme d'optimisation des tournées (Nearest Neighbor + Haversine)
real-time-tracking-service	8084	Suivi temps réel des visites, gestion des incidents
billing-reconciliation-service	8085	Facturation, prise en charge, rapprochement avec organismes

3. Pipeline du Projet

Étape 1 – Analyse et Conception

- Étude des besoins métier et identification des acteurs
- Identification des microservices et de leurs responsabilités (Single Responsibility Principle)
- Conception du diagramme d'architecture et des flux de données inter-services
- Définition des modèles de données : Patient, Caregiver, Planning, VisitStatus, Invoice

Étape 2 – Développement des Microservices

- Création des projets Spring Boot avec la structure en couches (Model → Repository → Service → Controller)
- Spring Data JPA + PostgreSQL pour la persistance des données (pattern Database per Service)
- DTOs avec validation Bean Validation (Jakarta) pour sécuriser les entrées
- Documentation automatique via SpringDoc OpenAPI / Swagger UI sur chaque service
- Lombok pour réduire le code boilerplate (getters, constructeurs, builders)

Étape 3 – Algorithme d'Optimisation des Tournées

> **Nearest Neighbor Heuristic (Algorithme du Plus Proche Voisin)**
> Approche greedy : à chaque itération, on sélectionne parmi les visites non encore planifiées celle dont le patient est géographiquement le plus proche du point courant du soignant. La **formule de Haversine** calcule la distance réelle entre deux coordonnées GPS sur la sphère terrestre. Complexité : **$O(n^2)$** – amplement suffisant pour des tournées de 5 à 25 visites par soignant par jour.

L'algorithme rend une route optimisée avec l'heure d'arrivée estimée pour chaque visite, la distance parcourue et la durée totale de la tournée.

Étape 4 – Conteneurisation et Infrastructure

- **Dockerfile multi-stage** pour chaque microservice : phase build (Maven 3.9 + JDK 17) → image finale légère (JRE Alpine)
- **docker-compose.yml** orchestrant tous les services, 3 bases PostgreSQL (patient-db, caregiver-db, billing-db), réseau Docker dédié `soins-network`
- Variables d'environnement externalisées (URL DB, URL Eureka) pour la portabilité dev/prod
- Volumes Docker pour la persistance des données entre les redémarrages

Étape 5 – Intégration et Tests

- Tests manuels complets via la **collection Postman** incluse (tous les endpoints CRUD + endpoints métier)
- Tests des health checks Actuator sur chaque service
- Vérification de la communication inter-services via l'API Gateway (routage par path)
- Validation du tableau de bord Eureka (enregistrement dynamique des 5 microservices)

4. Technologies et Outils

Catégorie	Technologie / Version	Justification
Framework Backend	Spring Boot 3.2	Développement rapide, auto-configuration, production-ready
Service Discovery	Spring Cloud Eureka	Enregistrement dynamique et découverte des services
API Gateway	Spring Cloud Gateway	Routage réactif, load balancing intégré
Persistance	Spring Data JPA + PostgreSQL 15	ORM robuste, requêtes déclaratives, migration DDL auto
Validation	Jakarta Bean Validation	Validation côté serveur des données entrantes
Documentation API	SpringDoc OpenAPI + Swagger UI	Documentation interactive générée automatiquement
Conteneurisation	Docker + Docker Compose	Déploiement reproductible, isolation complète
Langage	Java 17 (LTS)	Stabilité, performance, fonctionnalités modernes
Build	Apache Maven 3.9	Gestion des dépendances, lifecycle standardisé
Tests API	Postman	Tests manuels des endpoints REST, collection partageable

5. Gestion des Défis Métier

Gestion des imprévus

- Le *real-time-tracking-service* permet de signaler un incident (patient absent, urgence médicale) via `POST /tracking/visits/{id}/incident`
- Le statut de la visite est immédiatement mis à jour (INCIDENT ou ANNULEE) pour permettre une réaffectation à un autre soignant
- Toutes les mises à jour sont horodatées (`lastUpdated`) pour audit et traçabilité

Équité de charge entre soignants

- L'attribut `capaciteVisitesParJour` limite le nombre maximal de visites affectables à chaque soignant
- La liste des soignants disponibles (`GET /caregivers/available`) filtre automatiquement les soignants en congé ou indisponibles

Expérience patient

- Respect des fenêtres horaires préférées et des contraintes explicites du patient (matin uniquement, etc.)
- Sélection des soignants selon leur liste de compétences correspondant aux besoins cliniques du patient

6. Démarrage du Projet

```
# Lancer tous les services (première fois : téléchargement + build ~5-10 min)
docker-compose up --build

# Accès aux interfaces
# Eureka Dashboard  : http://localhost:8761
# API Gateway       : http://localhost:8080/api/patients
# Swagger Patient   : http://localhost:8081/swagger-ui.html
# Swagger Caregiver : http://localhost:8082/swagger-ui.html
# Swagger Visite    : http://localhost:8083/swagger-ui.html
# Swagger Tracking  : http://localhost:8084/swagger-ui.html
# Swagger Billing   : http://localhost:8085/swagger-ui.html
```